

Практичне заняття №6

Тема : **Шаблони.** Параметричний поліморфізм

Виконав: Шалівський Віталій

Група: alk-43

Київ 2026 р

Хіт роботи

У даній програмі створено шаблонний клас `SparseArray`, який використовується для подання розрідженого одновимірного масиву. Для зберігання даних застосовано контейнер `vector` стандартної бібліотеки STL, у якому елементи записуються у вигляді пар: індекс і значення.

Операція `operator[]` дозволяє звертатися до елементів за індексом. Якщо елемент із вказаним індексом уже існує, то повертається посилання на нього. Якщо такого елемента немає, то створюється новий елемент із початковим значенням типу `T`.

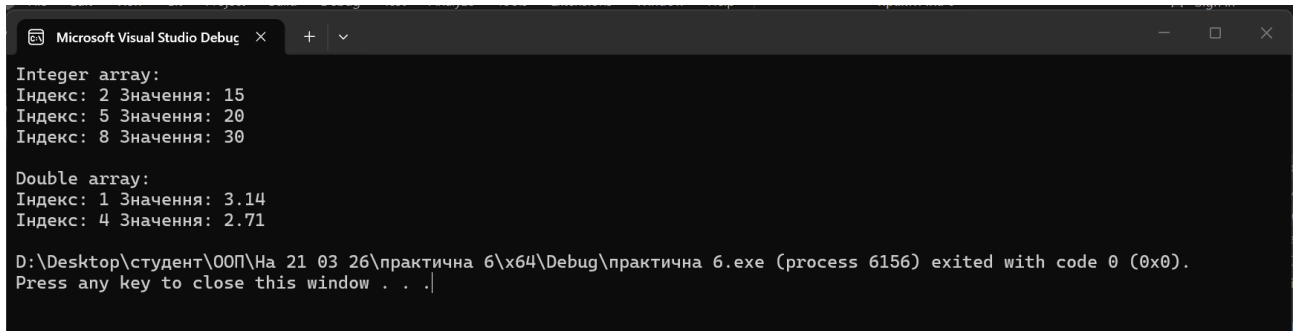
У функції `main()` показано використання шаблонного класу для типів `int` і `double`, що підтверджує універсальність шаблонів у мові C++.

Код програми

```
практична 6.cpp  X  What's New?
практична 6  (Global Scope)  main()

1  #include <iostream>
2  #include <vector>
3  #include <windows.h>
4  using namespace std;
5
6  // шаблонний клас розрідженого масиву
7  template <class T>
8  class SparseArray
9  {
10 private:
11     vector<pair<int, T>> data; // індекс і значення
12     int size;
13
14 public:
15     // конструктор
16     SparseArray(int s)
17     {
18         size = s;
19     }
20
21     // оператор індексації
22     T& operator[](int index)
23     {
24         for (auto& p : data)
25         {
26             if (p.first == index)
27                 return p.second;
28         }
29
30         // якщо елемент не знайдено - створимо новий
31         data.push_back({ index, T() });
32         return data.back().second;
33     }
34
35     // метод виведення елементів
36     void show()
37     {
38         for (auto& p : data)
39         {
40             cout << "Індекс: " << p.first << " Значення: " << p.second << endl;
41         }
42     };
43
44 int main()
45 {
46     SetConsoleCP(1251);
47     SetConsoleOutputCP(1251);
48
49     SparseArray<int> a(10);
50
51     a[2] = 15;
52     a[5] = 20;
53     a[8] = 30;
54
55     cout << "Integer array:" << endl;
56     a.show();
57
58     SparseArray<double> b(10);
59
60     b[1] = 3.14;
61     b[4] = 2.71;
62
63     cout << endl << "Double array:" << endl;
64     b.show();
65
66     return 0;
67 }
68
```

Результат виконання програми



```
Microsoft Visual Studio Debug
Integer array:
Індекс: 2 Значення: 15
Індекс: 5 Значення: 20
Індекс: 8 Значення: 30

Double array:
Індекс: 1 Значення: 3.14
Індекс: 4 Значення: 2.71

D:\Desktop\студент\00П\На 21 03 26\практична 6\x64\Debug\практична 6.exe (process 6156) exited with code 0 (0x0).
Press any key to close this window . . .|
```

Контрольні запитання та відповіді

1.Що таке родова функція?

Родова функція (template function) — це функція, яка може працювати з різними типами даних. Тип даних передається як параметр шаблону. Завдяки цьому одна і та сама функція може застосовуватися до різних типів даних.

2. Яка загальна форма родової функції?

Загальна форма: `template тип_даних функція(параметри) { ... }`

3.Чи може родова функція мати кілька родових аргументів?

Так, родова функція може мати декілька параметрів типів. Вони задаються через кому в операторі `template`.

4.Яка загальна форма родової функції з кількома родовими аргументами?

Загальна форма: `template тип функція(параметри) { ... }`

5. Яка відмінність між родовою та перевантаженою функціями?

Родові функції використовують шаблони і створюють одну універсальну функцію для різних типів. Перевантажені функції — це кілька різних функцій з однаковим ім'ям, але різними параметрами.

6. Яку версію генерує компілятор за наявності перевантаженої і родової функції?

Якщо існує явна перевантажена функція і родова функція, компілятор надає перевагу перевантаженій функції, оскільки вона є більш точною відповідністю.

7. Що таке родовий (параметризований) клас?

Родовий клас — це клас, у якому тип даних задається як параметр шаблону. Це дозволяє використовувати один і той самий клас для різних типів даних.

8. Яка загальна форма родового класу?

Загальна форма: `template class Ім'яКласу { ... };`

9. Яка загальна форма родового класу з кількома родовими типами?

Загальна форма: `template class Ім'яКласу { ... };`

10. Що таке параметричний поліморфізм?

Параметричний поліморфізм — це можливість використовувати одну функцію або клас для роботи з різними типами даних. У C++ він реалізується за допомогою шаблонів (templates).

11. Що таке контейнери? Наведіть приклади.

Контейнери — це класи, призначені для зберігання та організації даних. Приклади контейнерів: масив, список, стек, черга, дерево.

12. Які переваги використання родових класів?

Основні переваги:

- повторне використання коду;
- універсальність програм;
- зменшення дублювання коду;
- можливість працювати з різними типами даних.